DOCUMENTATION · PROJECT

Why QueryAPIGate

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=WHY. This copy was made on 4 November 2026.

Organizations often have valuable SQL sitting inside reports, BI dashboards, ETL pipelines, ad-hoc analysis and application code, with no path to a REST endpoint except writing a backend service around it.

```text
              SQL Query
                  |
                  v
          +------------------+
          |   QueryAPIGate   |
          |                  |
          | Parameters       |
          | SQL Guard        |
          | Permissions      |
          | Rate Limiting    |
          | Caching          |
          | Connection Pool  |
          | Streaming        |
          | Observability    |
          +--------+---------+
                   |
                   v
               REST API
                   |
          +--------+--------+
          v        v        v
        JSON      CSV      XLSX
```

If you already know SQL, you can produce a governed API without building an API application around it.

Best use cases

Internal data APIs. Expose internal data to web apps, mobile apps, internal tools and other engineering teams without building a service per query:

```http
GET /q/film_by_id?id=7
GET /q/example_top_films?top_n=5&category=Comedy
```

Reporting APIs. Turn an existing analytical SQL report into a reusable endpoint that dashboards or scheduled jobs can call directly, in JSON or as a CSV/XLSX export.

Partner and external-client integrations. Give a partner an API key scoped to a curated list of saved queries, with its own rate limit and expiry, instead of database credentials:

```text
Partner --API Key⟶ QueryAPIGate --queries grant⟶ Database
```

Data engineering to application engineering. Data teams write and version the SQL; application teams consume it as a normal REST endpoint, without either side depending on the other's deploy cycle.

Data export. Stream large results as CSV, TSV, XLSX, JSON or NDJSON without loading the whole result into memory first.

Rapid prototyping. Get a working, parameterized API around an existing query for a proof of concept or internal tool without standing up a backend application for it.

How QueryAPIGate differs

APPROACH	PRIMARY MODEL
QueryAPIGate	**SQL query -> governed REST API**
PostgREST	PostgreSQL schema -> REST API
Hasura	Database -> GraphQL/API platform
DreamFactory	Data source -> generated APIs
SQLPad	SQL -> interactive query environment

QueryAPIGate's primary abstraction is the SQL query as an API resource, not the schema as a whole - so it fits well when an API should expose one specific, curated query rather than automatically surface an entire database.

Philosophy

Your database already contains the data logic. The goal isn't to replace application backends; it's to remove repetitive boilerplate when the actual requirement is "expose this query safely as an API" - existing SQL, plus governance, is the API.